

TASK - 10

(dashboard creation)

TASK :

Two Esp32s were used each will generate 5 random telemetries such as temperature, lighting, moisture etc.... Data from both esp32s were sent to the laptop which acts as a server using mqtt protocol. Here we store the present values as well as the previous value. The present value is stored inside state file and previous were stored in log file. also we want to see this in a dashboard

To build this frontend , we can use a library called **Streamlit**. It is an incredibly popular, open-source Python framework that lets you build interactive, real-time web dashboards entirely in Python - no HTML, CSS, or JavaScript required.

The Frontend Architecture Concept

system will now work like this:

1. Background MQTT script (receiver_with_storage.py) keeps running, fetching data from the ESPs, and saving it to state.json and the folders.
2. New Streamlit Frontend script will run at the same time. It reads that live state.json file and transforms the numbers into clean visual charts, battery bars, and colorful parameter cards.

Step 1: Install Streamlit in VS Code

1. Open terminal in VS Code.
2. Stop current MQTT script for a brief moment by pressing Ctrl + C.
3. Type the following command to install Streamlit and click Enter:

```
pip install streamlit
```

Step 1: Upgrading the Backend Script

(receiver_with_storage.py)

```
import paho.mqtt.client as mqtt
import json
import os
from datetime import datetime

MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883
TOPIC_ESP1 = "group_project/esp1"
```

```

TOPIC_ESP2 = "group_project/esp2"

BASE_DIR = "telemetry_data"
STATE_FILE = os.path.join(BASE_DIR, "state.json")
LOGS_FOLDER = os.path.join(BASE_DIR, "logs")

def ensure_directories_exist():
    os.makedirs(LOGS_FOLDER, exist_ok=True)

def save_nested_folder_log(esp_node, current_data):
    ensure_directories_exist()
    now = datetime.now()
    timestamp_str = now.strftime("%Y-%m-%d %H:%M:%S")

    # NEW ID FORMAT: Human-readable timestamp ID (HH-MM-SS)
    time_id = f"Time_{now.strftime('%H-%M-%S')}"
    node_folder_name = "ESP32_NODE_1" if esp_node == "esp1" else
"ESP32_NODE_2"

    # Construct paths
    target_folder_path = os.path.join(LOGS_FOLDER, time_id,
node_folder_name)
    os.makedirs(target_folder_path, exist_ok=True)

    node_index = "1" if esp_node == "esp1" else "2"
    structured_telemetry = {
        "Timestamp": timestamp_str,
        "Temperature": current_data.get('temp', 0.0),
        "Distance": current_data.get('dist', 0.0),
        "Moisture": current_data.get('moist', 0.0),
        "Light": current_data.get('light', 0.0),
        "Battery": current_data.get('batt', 0.0)
    }

    # Save historical log
    data_filename = os.path.join(target_folder_path,
f"node{node_index}_telemetry.json")
    with open(data_filename, "w") as f:
        json.dump(structured_telemetry, f, indent=4)

    # Update state.json for the live dashboard view
    state_data = {}
    if os.path.exists(STATE_FILE) and os.path.getsize(STATE_FILE) > 0:
        try:
            with open(STATE_FILE, "r") as f:
                state_data = json.load(f)
        except (json.JSONDecodeError, FileNotFoundError):
            pass

```

```

state_data[esp_node] = {
    "last_updated": timestamp_str,
    "parameters": current_data
}
with open(STATE_FILE, "w") as f:
    json.dump(state_data, f, indent=4)

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("[*] Connected to Broker!")
        client.subscribe(TOPIC_ESP1)
        client.subscribe(TOPIC_ESP2)
    else:
        print("[-] Connection failed")

def on_message(client, userdata, msg):
    try:
        payload_text = msg.payload.decode("utf-8")
        data = json.loads(payload_text)
        node = "esp1" if msg.topic == TOPIC_ESP1 else "esp2"
        save_nested_folder_log(node, data)
    except Exception as e:
        print(f"[-] Error: {e}")

ensure_directories_exist()
client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message
client.connect(MQTT_BROKER, MQTT_PORT, 60)

try:
    client.loop_forever()
except KeyboardInterrupt:
    client.disconnect()

```

Step 3: Create the Dashboard Code

Inside IoT_Project folder, create a brand-new file named dashboard.py and paste the following code:

```

import streamlit as st
import json
import os
import pandas as pd
import glob

# Page layout configurations

```

```

st.set_page_config(page_title="Classic IoT Control Center",
page_icon="🌿", layout="wide")

# 🍷 Classic Theme Injection: Dark Olive Green (#556B2F) and Clean
White/Off-White
st.markdown("""
    <style>
    .main { background-color: #FFFFFF; color: #333333; }
    h1 { color: #556B2F !important; font-family: 'Georgia', serif;
font-weight: bold; }
    h2 { color: #556B2F !important; font-family: 'Arial', sans-serif;
}
    .stSelectbox label { color: #556B2F !important; font-weight: bold;
}
    div[data-testid="stBlock"] {
        border: 2px solid #556B2F;
        padding: 20px;
        border-radius: 10px;
        background-color: #FDFDFD;
        box-shadow: 2px 2px 10px rgba(85,107,47,0.1);
    }
    </style>
""", unsafe_style_with_html=True)

STATE_FILE = os.path.join("telemetry_data", "state.json")
LOGS_FOLDER = os.path.join("telemetry_data", "logs")

def get_live_state():
    if os.path.exists(STATE_FILE) and os.path.getsize(STATE_FILE) > 0:
        try:
            with open(STATE_FILE, "r") as f: return json.load(f)
        except: return {}
    return {}

def get_all_time_ids():
    if not os.path.exists(LOGS_FOLDER): return []
    # Fetch directory names inside logs/ folder
    dirs = [d for d in os.listdir(LOGS_FOLDER) if
os.path.isdir(os.path.join(LOGS_FOLDER, d))]
    return sorted(dirs, reverse=True)

def get_historical_temp_data(node_folder_name, file_pattern):
    """Searches through all time-ID folders to aggregate temperature
values sequentially."""
    temp_records = []
    if not os.path.exists(LOGS_FOLDER): return pd.DataFrame()

    # Find all nested telemetry files matching the node

```

```

    search_path = os.path.join(LOGS_FOLDER, "*", node_folder_name,
file_pattern)
    file_list = sorted(glob.glob(search_path))

    for filepath in file_list:
        try:
            with open(filepath, "r") as f:
                file_data = json.load(f)
                temp_records.append({
                    "Timestamp": file_data.get("Timestamp",
"Unknown"),
                    "Temperature (°C)":
float(file_data.get("Temperature", 0.0))
                })
        except:
            pass
    return pd.DataFrame(temp_records)

# --- HEADER SECTION ---
st.title("🏠 Local Cloud IoT Operations Dashboard")
st.write("Academic Multi-Node Monitoring Framework – Designed in
Classic Dark Olive Green & White.")
st.markdown("---")

live_data = get_live_state()
available_ids = get_all_time_ids()

# Create the two side-by-side control boxes requested by your guide
col1, col2 = st.columns(2)

# ===== ESP32 NODE 1 BOX =====
with col1:
    st.header("💠 Station Location 1 (ESP32 Node 1)")
    view_option_1 = st.selectbox(
        "Select Data Stream to Display (Node 1):",
        ["State (Present Value)", "Log (Historical Records)",
"Temperature Log Graph"],
        key="node1_view"
    )

    if view_option_1 == "State (Present Value)":
        if "esp1" in live_data:
            p1 = live_data["esp1"]["parameters"]
            st.caption(f"Last ping received:
{live_data['esp1']['last_updated']}")
            # Construct formatted DataFrame for clean table view
            df1 = pd.DataFrame({
                "Parameter Metric": ["🌡️ Temperature", "📏 Distance",

```

```

"💧 Moisture", "☀️ Light Input", "🔋 Battery Capacity"],
    "Current Present Value": [f"{p1.get('temp')} °C",
f"{p1.get('dist')} cm", f"{p1.get('moist')} %", f"{p1.get('light')} %", f"{p1.get('batt')} %"]
    })
    st.table(df1)
else:
    st.warning("No live data registered for Node 1 yet.")

elif view_option_1 == "Log (Historical Records)":
    if available_ids:
        selected_id_1 = st.selectbox("Select Target Unique Time ID Slot:", available_ids, key="node1_id")
        target_path = os.path.join(LOGS_FOLDER, selected_id_1, "ESP32_NODE_1", "node1_telemetry.json")
        if os.path.exists(target_path):
            with open(target_path, "r") as f:
                log_content = json.load(f)
            st.json(log_content)
        else:
            st.info("No logs saved by Node 1 during this precise time frame.")
    else:
        st.info("No history log folders found yet.")

elif view_option_1 == "Temperature Log Graph":
    df_chart1 = get_historical_temp_data("ESP32_NODE_1", "node1_telemetry.json")
    if not df_chart1.empty:
        st.write("📈 **Continuous Temperature Trend Analysis (t1):**")
        st.line_chart(data=df_chart1, x="Timestamp", y="Temperature (°C)")
    else:
        st.info("Insufficient timeline data points to draw graph trends.")

# ===== ESP32 NODE 2 BOX =====
with col2:
    st.header("📍 Station Location 2 (ESP32 Node 2)")
    view_option_2 = st.selectbox(
        "Select Data Stream to Display (Node 2):",
        ["State (Present Value)", "Log (Historical Records)", "Temperature Log Graph"],
        key="node2_view"
    )

```

```

if view_option_2 == "State (Present Value)":
    if "esp2" in live_data:
        p2 = live_data["esp2"]["parameters"]
        st.caption(f"Last ping received:
{live_data['esp2']['last_updated']}")
        df2 = pd.DataFrame({
            "Parameter Metric": ["🌡️ Temperature", "📏 Distance",
"💧 Moisture", "☀️ Light Input", "🔋 Battery Capacity"],
            "Current Present Value": [f"{p2.get('temp')} °C",
f"{p2.get('dist')} cm", f"{p2.get('moist')} %", f"{p2.get('light')}
%", f"{p2.get('batt')} %"]
        })
        st.table(df2)
    else:
        st.warning("No live data registered for Node 2 yet.")

elif view_option_2 == "Log (Historical Records)":
    if available_ids:
        selected_id_2 = st.selectbox("Select Target Unique Time ID
Slot:", available_ids, key="node2_id")
        target_path = os.path.join(LOGS_FOLDER, selected_id_2,
"ESP32_NODE_2", "node2_telemetry.json")
        if os.path.exists(target_path):
            with open(target_path, "r") as f:
                log_content = json.load(f)
                st.json(log_content)
        else:
            st.info("No logs saved by Node 2 during this precise
time frame.")
    else:
        st.info("No history log folders found yet.")

elif view_option_2 == "Temperature Log Graph":
    df_chart2 = get_historical_temp_data("ESP32_NODE_2",
"node2_telemetry.json")
    if not df_chart2.empty:
        st.write("📈 **Continuous Temperature Trend Analysis
(t2):**")
        st.line_chart(data=df_chart2, x="Timestamp",
y="Temperature (°C)")
    else:
        st.info("Insufficient timeline data points to draw graph
trends.")

# Refresh button to trigger updates manually
if st.button("🔄 Force Refresh Dashboard View"):
    st.rerun()

```

Step 3: How to Run the Frontend System

Because you are running a complete server setup now, you need **two terminal tabs** open in VS Code at the same time:

1. **Terminal Tab 1 (The Backend Receiver):** Run MQTT file logger script so it can gather information:
`python receiver_with_storage.py`
2. **Terminal Tab 2 (The Frontend Dashboard):** Click the + sign in the VS Code terminal window to create a separate, second terminal window. Inside it, launch your frontend app using this special command:

```
python -m streamlit run dashboard.py
```

Step 1: Complete the Terminal Prompt

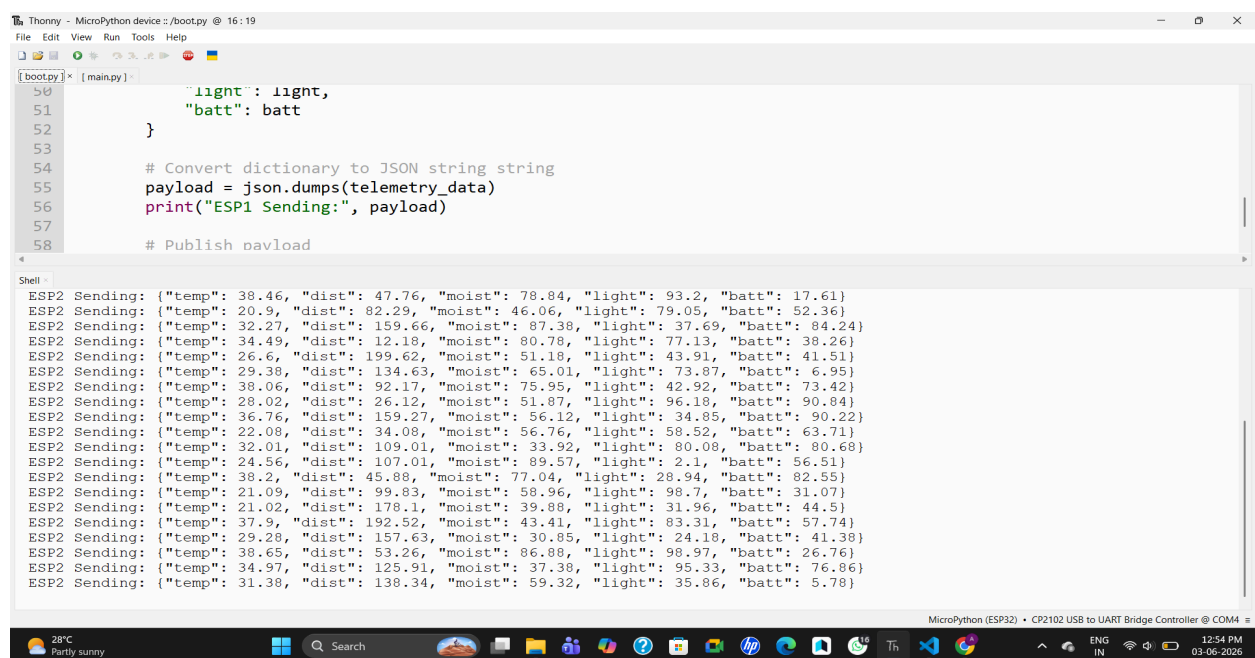
Click your mouse cursor directly inside that terminal window where it says Email: so you can type.

Do not type anything. Just press the Enter key on your keyboard to leave it blank.

If it asks you a second question about subscribing to a newsletter, just type n and press Enter.

In the terminal tab a site link will be shown, copy it and paste it in a web browser.

OUTPUT THONNY

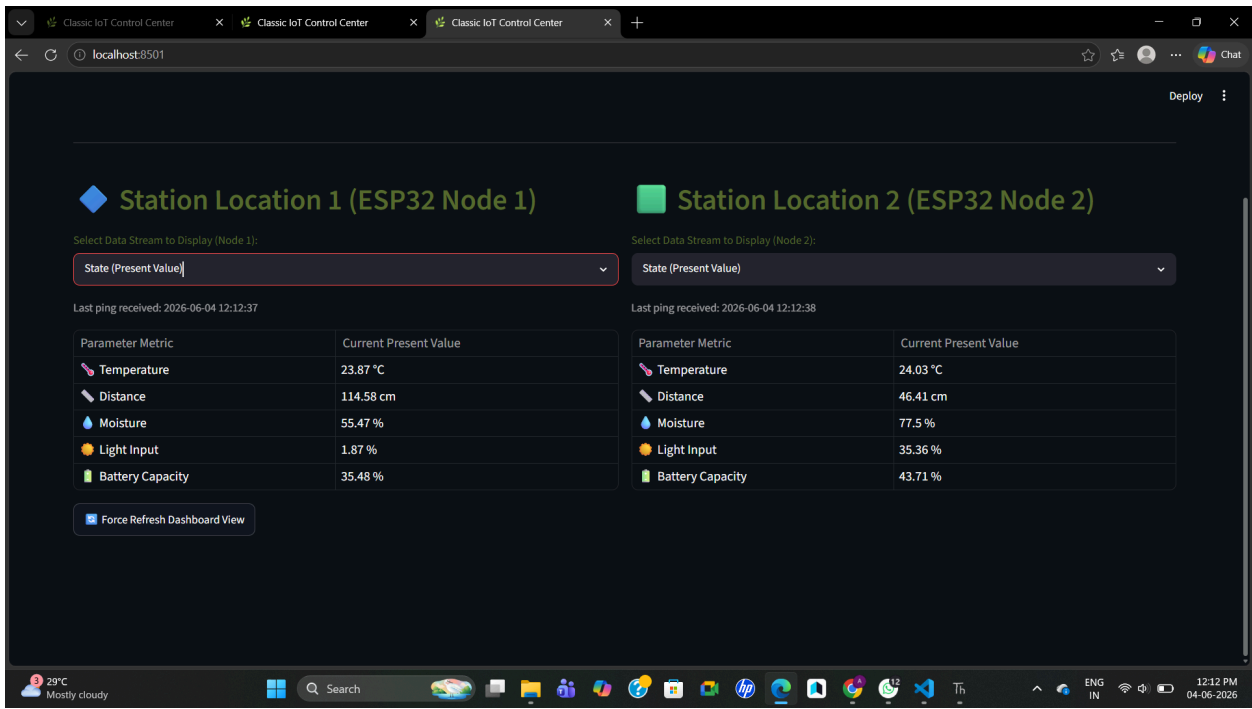
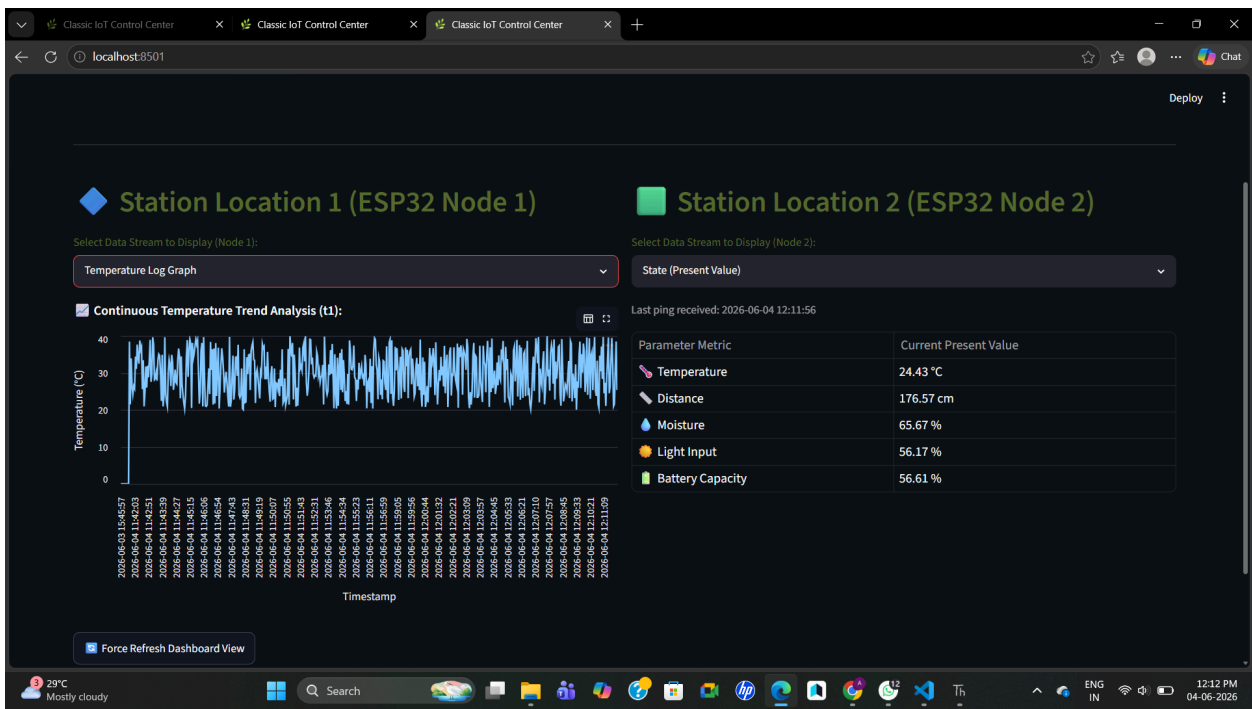


The screenshot shows the Thonny IDE interface. The top part displays a Python script in a file named `main.py`. The script defines a dictionary `telemetry_data` with keys for `temp`, `dist`, `moist`, `light`, and `batt`. It then converts this dictionary to a JSON string using `json.dumps` and prints it with the prefix `"ESP1 Sending:"`. The bottom part of the window shows the Shell output, which displays a series of JSON strings, each preceded by `ESP2 Sending:`. The data values vary across the different sends. The status bar at the bottom indicates the device is a MicroPython (ESP32) connected via CP2102 USB to UART Bridge Controller @ COM4.

```
50         "light": light,
51         "batt": batt
52     }
53
54     # Convert dictionary to JSON string
55     payload = json.dumps(telemetry_data)
56     print("ESP1 Sending:", payload)
57
58     # Publish payload
```

```
ESP2 Sending: {"temp": 38.46, "dist": 47.76, "moist": 78.84, "light": 93.2, "batt": 17.61}
ESP2 Sending: {"temp": 20.9, "dist": 82.29, "moist": 46.06, "light": 79.05, "batt": 52.36}
ESP2 Sending: {"temp": 32.27, "dist": 159.66, "moist": 87.38, "light": 37.69, "batt": 84.24}
ESP2 Sending: {"temp": 34.49, "dist": 12.18, "moist": 80.78, "light": 77.13, "batt": 38.26}
ESP2 Sending: {"temp": 26.6, "dist": 199.62, "moist": 51.18, "light": 43.91, "batt": 41.51}
ESP2 Sending: {"temp": 29.38, "dist": 134.63, "moist": 65.01, "light": 73.87, "batt": 6.95}
ESP2 Sending: {"temp": 38.06, "dist": 92.17, "moist": 75.95, "light": 42.92, "batt": 73.42}
ESP2 Sending: {"temp": 28.02, "dist": 26.12, "moist": 51.87, "light": 96.18, "batt": 90.84}
ESP2 Sending: {"temp": 36.76, "dist": 159.27, "moist": 56.12, "light": 34.85, "batt": 90.22}
ESP2 Sending: {"temp": 22.08, "dist": 34.08, "moist": 56.76, "light": 58.52, "batt": 63.71}
ESP2 Sending: {"temp": 32.01, "dist": 109.01, "moist": 33.92, "light": 80.08, "batt": 80.68}
ESP2 Sending: {"temp": 24.56, "dist": 107.01, "moist": 89.57, "light": 2.1, "batt": 56.51}
ESP2 Sending: {"temp": 38.2, "dist": 45.88, "moist": 77.04, "light": 28.94, "batt": 82.55}
ESP2 Sending: {"temp": 21.09, "dist": 99.83, "moist": 58.96, "light": 98.7, "batt": 31.07}
ESP2 Sending: {"temp": 21.02, "dist": 178.1, "moist": 39.88, "light": 31.96, "batt": 44.5}
ESP2 Sending: {"temp": 37.9, "dist": 192.52, "moist": 43.41, "light": 83.31, "batt": 57.74}
ESP2 Sending: {"temp": 29.28, "dist": 157.63, "moist": 30.85, "light": 24.18, "batt": 41.38}
ESP2 Sending: {"temp": 38.65, "dist": 53.26, "moist": 86.88, "light": 98.97, "batt": 26.76}
ESP2 Sending: {"temp": 34.97, "dist": 125.91, "moist": 37.38, "light": 95.33, "batt": 76.86}
ESP2 Sending: {"temp": 31.38, "dist": 138.34, "moist": 59.32, "light": 35.86, "batt": 5.78}
```


DASHBOARD



Classic IoT Control Center

Classic IoT Control Center

Classic IoT Control Center

+

localhost:8501

Deploy

Station Location 1 (ESP32 Node 1)

Select Data Stream to Display (Node 1):

Log (Historical Records)

Select Target Unique Time ID Slot:

log_20260603_1554

```
{
  "Timestamp" : "2026-06-03 15:54:46"
  "Temperature_t1" : "24.76 °C"
  "Distance_d1" : "84.89 cm"
  "Moisture_m1" : "54.71 %"
  "Light_l1" : "40.93 %"
  "Battery_b1" : "40.56 %"
}
```

Force Refresh Dashboard View

Station Location 2 (ESP32 Node 2)

Select Data Stream to Display (Node 2):

State (Present Value)

Last ping received: 2026-06-04 12:13:02

Parameter Metric	Current Present Value
Temperature	35.31 °C
Distance	119.0 cm
Moisture	48.72 %
Light Input	71.86 %
Battery Capacity	32.5 %

29°C Mostly cloudy

Search

ENG IN

12:13 PM 04-06-2026

Classic IoT Control Center

Classic IoT Control Center

Classic IoT Control Center

+

localhost:8501

Deploy

Station Location 1 (ESP32 Node 1)

Select Data Stream to Display (Node 1):

Log (Historical Records)

Select Target Unique Time ID Slot:

log_20260603_1554

log_20260603_1554

log_20260603_1553

log_20260603_1552

log_20260603_1551

log_20260603_1550

log_20260603_1549

log_20260603_1548

Station Location 2 (ESP32 Node 2)

Select Data Stream to Display (Node 2):

State (Present Value)

Last ping received: 2026-06-04 12:13:02

Parameter Metric	Current Present Value
Temperature	35.31 °C
Distance	119.0 cm
Moisture	48.72 %
Light Input	71.86 %
Battery Capacity	32.5 %

29°C Mostly cloudy

Search

ENG IN

12:13 PM 04-06-2026

Classic IoT Control Center

Classic IoT Control Center

Classic IoT Control Center

+

localhost:8501

Deploy

Station Location 1 (ESP32 Node 1)

Select Data Stream to Display (Node 1):

Log (Historical Records)

Select Target Unique Time ID Slot:

log_20260603_1553

```
{
  "Timestamp" : "2026-06-03 15:53:58"
  "Temperature_t1" : "26.03 °C"
  "Distance_d1" : "151.01 cm"
  "Moisture_m1" : "65.31 %"
  "Light_l1" : "65.81 %"
  "Battery_b1" : "45.26 %"
}
```

Force Refresh Dashboard View

Station Location 2 (ESP32 Node 2)

Select Data Stream to Display (Node 2):

State (Present Value)

Last ping received: 2026-06-04 12:13:14

Parameter Metric	Current Present Value
Temperature	38.54 °C
Distance	76.53 cm
Moisture	85.47 %
Light Input	52.44 %
Battery Capacity	27.88 %

29°C Mostly cloudy

Search

ENG IN

12:13 PM 04-06-2026

Classic IoT Control Center

Classic IoT Control Center

Classic IoT Control Center

+

localhost:8501

Deploy

Station Location 1 (ESP32 Node 1)

Select Data Stream to Display (Node 1):

Log (Historical Records)

Select Target Unique Time ID Slot:

log_20260603_1553

```
{
  "Timestamp" : "2026-06-03 15:53:58"
  "Temperature_t1" : "26.03 °C"
  "Distance_d1" : "151.01 cm"
  "Moisture_m1" : "65.31 %"
  "Light_l1" : "65.81 %"
  "Battery_b1" : "45.26 %"
}
```

Force Refresh Dashboard View

Station Location 2 (ESP32 Node 2)

Select Data Stream to Display (Node 2):

Log (Historical Records)

Select Target Unique Time ID Slot:

log_20260603_1554

```
{
  "Timestamp" : "2026-06-03 15:54:47"
  "Temperature_t2" : "22.3 °C"
  "Distance_d2" : "98.41 cm"
  "Moisture_m2" : "83.78 %"
  "Light_l2" : "77.73 %"
  "Battery_b2" : "73.26 %"
}
```

29°C Mostly cloudy

Search

ENG IN

12:14 PM 04-06-2026

